Ensemble Learning & Support Vector Machines

STAT3612: Statistical Machine Learning

Yinghao Zhu

HKU

November 2026

Outline & Companion Notebook

1. Part 1: Ensemble Learning
   - Bagging & Random Forests
   - Boosting: AdaBoost, GBDT & XGBoost
2. Part 2: Support Vector Machines (SVM)
   - Maximum Margin & Support Vectors
   - Kernel Comparison on Iris & XOR

Hands-On Companion: `ensemble_svm_tutorial.ipynb`

- **Part 1 (Ensemble Learning on `make_moons`):** Single Decision Tree baseline → Bagging → Random Forest → AdaBoost, Gradient Boosting, and XGBoost.
- **Part 2 (Support Vector Machines on `Iris` & `XOR`):** Maximum-margin hyperplane, support vectors, and kernel comparison (`linear`, `poly`, `rbf`, `sigmoid`).

Why Ensemble Learning? Bagging vs. Boosting

Core Idea

Combine predictions of multiple base estimators $h_1, \ldots, h_B$ to achieve better generalization and robustness than any single decision tree.

1. Bagging (Bootstrap Aggregating)

- Train deep, unpruned trees independently in parallel on bootstrap subsets (sampling with replacement).
- Aggregate via majority vote / averaging:

$$\hat{f}_{\text{bag}}(\mathbf{x}) = \frac{1}{B} \sum_{b=1}^{B} h_b(\mathbf{x})$$

- **Goal:** Reduce Variance of unstable, high-variance trees (Bagging, Random Forest).

2. Boosting

- Train shallow weak learners sequentially, where each new tree corrects errors of previous ones:

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \eta\, h_m(\mathbf{x})$$

- Uses simple base learners (stumps or depth-3 trees) with high bias and low variance.
- **Goal:** Reduce Bias (AdaBoost, Gradient Boosting, XGBoost).

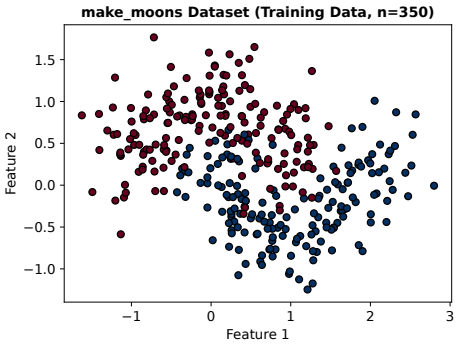
Hands-On Setup: make_moons & Boundary Plotting

1. Generate Non-Linear Noisy Data:

```
 from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.inspection import DecisionBoundaryDisplay
X, y = make_moons(n_samples=500, noise=0.3,
random_state=42)
X_train, X_test, y_train, y_test = train_test_split(
  X, y, test_size=0.3, random_state=42)
```

2. Reusable Boundary Helper:

```
def plot_decision_boundary(clf, X, y, ax, title):
 DecisionBoundaryDisplay.from_estimator(
   clf, X, response_method="predict",
   cmap=plt.cm.RdBu, alpha=0.5, ax=ax)
 ax.scatter(X[:, 0], X[:, 1], c=y,
   cmap=plt.cm.RdBu, edgecolors="k")
 ax.set_title(title)
```



- 350 train / 150 test points; `noise` $= 0.3$ penalizes overfitting.

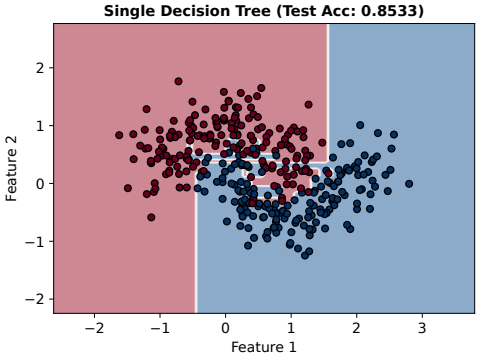
Baseline: A Single Unrestricted Decision Tree

Core Code (`DecisionTreeClassifier`):

```
 from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
# No max_depth limit -> grows until pure
tree_clf = DecisionTreeClassifier(random_state=42)
tree_clf.fit(X_train, y_train)
y_pred_tree = tree_clf.predict(X_test)
tree_accuracy = accuracy_score(y_test, y_pred_tree)
# Single Decision Tree Accuracy:  0.8533
```

What We Observe:

- Recursive axis-aligned splits ($x_j \leq t$) create intricate, jagged rectangles around isolated noisy training points.
- Classic overfitting (high variance): 100% training accuracy, but only **0.8533** test accuracy.



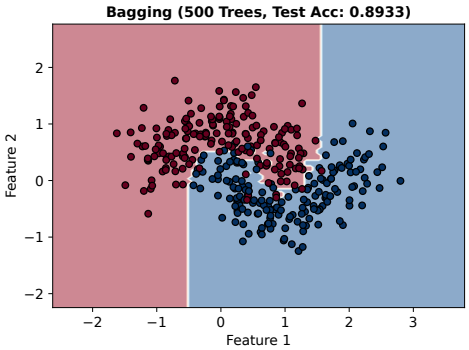
Bagging: Bootstrap Aggregating (BaggingClassifier)

Core Code (500 Bootstrap Trees):

```
from sklearn.ensemble import BaggingClassifier
bag_clf = BaggingClassifier(
  DecisionTreeClassifier(random_state=42),
  n_estimators=500, max_samples=1.0,
  bootstrap=True, n_jobs=-1, random_state=42)
bag_clf.fit(X_train, y_train)
bag_accuracy = accuracy_score(y_test,
bag_clf.predict(X_test))
# Bagging Classifier Accuracy:  0.8933
```

Why It Works:

- Each of the 500 trees sees a different bootstrap sample ($\approx 63.2\%$ unique training rows).
- Majority voting cancels out single-tree noise $\Longrightarrow$ smoother boundary, jumping to **0.8933** ($+4.0\%$).



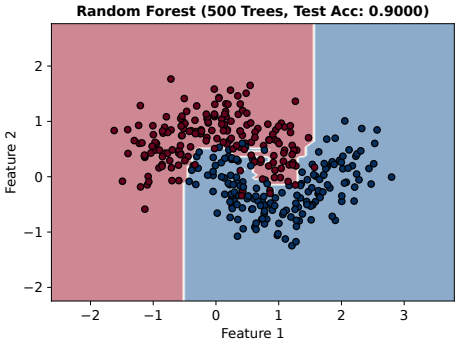
Random Forest: Decorrelating Trees (`RandomForestClassifier`)

Core Code:

```
from sklearn.ensemble import RandomForestClassifier
# Bootstrap rows + random feature subset per split
rnd_clf = RandomForestClassifier(
  n_estimators=500, n_jobs=-1, random_state=42)
rnd_clf.fit(X_train, y_train)
rf_accuracy = accuracy_score(y_test,
rnd_clf.predict(X_test))
# Random Forest Accuracy:  0.9000
```

How Random Forest Improves on Bagging:

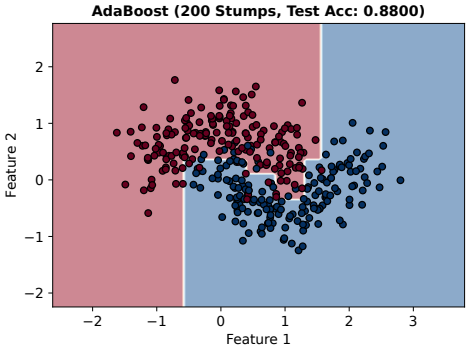
- Variance of B correlated trees (Corr $= \rho$): $\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$.
- Sampling a random subset of features (`max_features`, default $\sqrt{p}$) at each split decorrelates trees (lowers ρ), reaching **0.9000**.



Boosting I: AdaBoost (AdaBoostClassifier)

Core Code (200 Decision Stumps):

```
from sklearn.ensemble import AdaBoostClassifier
# Base learner: shallow stump (max_depth=1)
ada_clf = AdaBoostClassifier(
  DecisionTreeClassifier(max_depth=1,
random_state=42),
  n_estimators=200, learning_rate=0.5,
random_state=42)
ada_clf.fit(X_train, y_train)
ada_accuracy = accuracy_score(y_test,
ada_clf.predict(X_test))
# AdaBoost Classifier Accuracy: 0.8800
```



Mechanism:

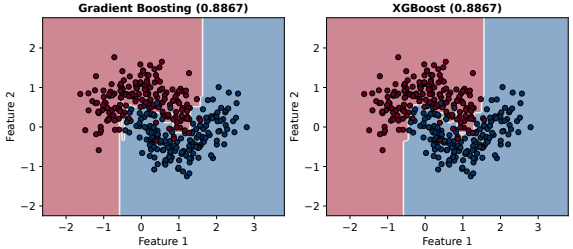
- Sequentially fits decision stumps (`max_depth` $= 1$).
- Up-weights misclassified samples $w_i^{(t+1)}$ and combines stumps by weighted vote $\sum_t \alpha_t h_t(\mathbf{x})$ (**0.8800**).

Boosting II: Gradient Boosting (GBDT) & XGBoost

Core Code:

```
 from sklearn.ensemble import
GradientBoostingClassifier
import xgboost as xgb
# 1.  Gradient Boosting (fits residual errors)
gbrt_clf = GradientBoostingClassifier(
  n_estimators=100, learning_rate=0.1,
max_depth=3,
  random_state=42).fit(X_train, y_train)
# 2.  XGBoost (regularized + parallelized GBDT)
xgb_clf = xgb.XGBClassifier(
  n_estimators=100, learning_rate=0.1,
max_depth=3,
  eval_metric="logloss",
random_state=42).fit(X_train, y_train)
```

- **GBDT:** Fits pseudo-residuals $-\partial\mathcal{L}/\partial F(\mathbf{x}_i)$.
- **XGBoost:** 2nd-order gradients + penalty $\Omega(h_m)$.



- Both achieve **0.8867** test accuracy on `make_moons`.

Ensemble Summary & Assignment 3 Guide (Part 1)

Test Accuracy Summary (`results_df`):

Model	Test Acc.
Random Forest	**0.9000**
Bagging (500 Trees)	0.8933
XGBoost (`max_depth` = 3)	0.8867
Gradient Boosting (`max_depth` = 3)	0.8867
AdaBoost (200 Stumps)	0.8800
Single Decision Tree	0.8533

- **Takeaway:** Every ensemble method beats the single overfitted tree (+2.7% to +4.7%).

Assignment 3 (Q1–Q2) Code Pattern:

```
from sklearn.tree import plot_tree
# Q1(c-d): Entropy tree + visualization
dt = DecisionTreeClassifier(
 max_depth=3, criterion="entropy")
dt.fit(X_train, y_train); plot_tree(dt, filled=True)
# Q2(a-b): Sweep max_depth = 2..12
for d in range(2, 13):
 t = DecisionTreeClassifier(max_depth=d,
random_state=2025)
 rf = RandomForestClassifier(n_estimators=80,
  max_features=4, max_depth=d, random_state=2025)
```

- **Why cap depth?** Single-tree test accuracy peaks early then drops; RF stays higher and more stable!

Support Vector Machines: Margins, Support Vectors & Kernels

- **Soft-Margin SVM Primal Objective:**

$$\min_{\mathbf{w},b,\boldsymbol{\xi}} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n}\xi_i \quad \text{s.t.} \quad y_i(\mathbf{w}^T\phi(\mathbf{x}_i) + b) \geq 1 - \xi_i,\ \xi_i \geq 0$$

- **Geometry in `DecisionBoundaryDisplay` (`levels=[-1, 0, 1]`):**
  - **Solid line ($f(\mathbf{x}) = 0$):** Decision boundary.
  - **Dashed lines ($f(\mathbf{x}) = \pm 1$):** Margin boundaries (street width $\frac{2}{\|\mathbf{w}\|}$).
  - **Circled points (`clf.support_vectors_`):** Training samples on or inside the margin ($y_i f(\mathbf{x}_i) \leq 1$). Only support vectors determine the boundary!
- **The Kernel Trick ($K(\mathbf{x}_i, \mathbf{x}_j) = \langle\phi(\mathbf{x}_i), \phi(\mathbf{x}_j)\rangle$):**
  - Computes inner products in a transformed feature space without explicitly computing $\phi(\mathbf{x})$, producing curved boundaries in the original 2D space.

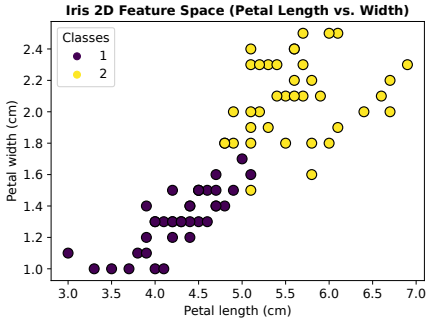
Hands-On Setup: Iris 2D Feature Space & SVC Helper

1. Select 2 Features & 2 Classes from Iris ($N = 100$):

```
 data = load_iris()
X = data.data[:, [2, 3]] # petal length & width
y = data.target
idx = np.where((y == 1) | (y == 2)) # versicolor vs
virginica
X, y = X[idx], y[idx]
X_train, X_test, y_train, y_test = train_test_split(
  X, y, random_state=42, test_size=0.25, stratify=y)
```

2. Plot Boundary, Margins & Support Vectors:

```
 clf = svm.SVC(kernel=kernel, gamma=2).fit(X, y)
DecisionBoundaryDisplay.from_estimator(clf, X, ax=ax,
  response_method="decision_function",
  plot_method="contour", levels=[-1, 0, 1],
  colors=["k", "k", "k"], linestyles=["-", "-", "-"])
ax.scatter(clf.support_vectors_[:, 0],
clf.support_vectors_[:, 1],
  s=150, facecolors="none", edgecolors="k")
```



- Classes 1 and 2 nearly separate by petal length/width, with overlap near $(4.8, 1.6)$.

Linear vs. Polynomial Kernel on Iris

1. Linear Kernel (Test Acc = 0.88):

$$K(\mathbf{x}_1, \mathbf{x}_2) = \mathbf{x}_1^T \mathbf{x}_2$$

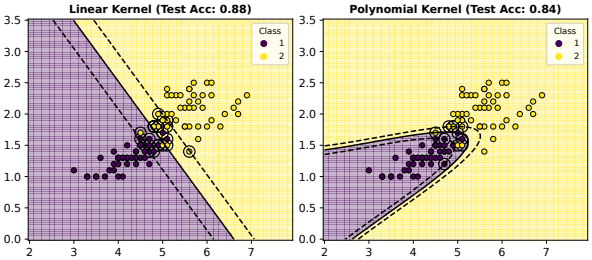
```
mmc_clf = SVC(kernel="linear",
random_state=42)
mmc_clf.fit(X_train, y_train)
mmc_clf.score(X_test, y_test) # 0.88
```

2. Polynomial Kernel (Test Acc = 0.84):

$$K(\mathbf{x}_1, \mathbf{x}_2) = (\gamma \mathbf{x}_1^T \mathbf{x}_2 + r)^d$$

```
mmc_clf = SVC(kernel="poly", random_state=42)
mmc_clf.fit(X_train, y_train)
mmc_clf.score(X_test, y_test) # 0.84
```

- Degree $d = 3$ curves the boundary and narrows the margin around the overlap zone.



RBF vs. Sigmoid Kernel on Iris

3. RBF / Gaussian Kernel (Test Acc = 0.92):

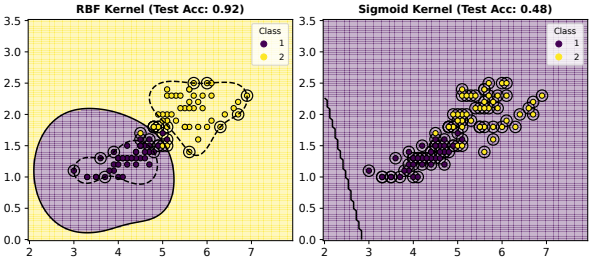
$$K(\mathbf{x}_1, \mathbf{x}_2) = \exp\left(-\gamma \|\mathbf{x}_1 - \mathbf{x}_2\|^2\right)$$

```
mmc_clf = SVC(kernel="rbf", random_state=42)
mmc_clf.fit(X_train, y_train)
mmc_clf.score(X_test, y_test) # 0.92 (Best!)
```

4. Sigmoid Kernel (Test Acc = 0.48):

$$K(\mathbf{x}_1, \mathbf{x}_2) = \tanh(\gamma \mathbf{x}_1^T \mathbf{x}_2 + r)$$

```
mmc_clf = SVC(kernel="sigmoid",
random_state=42)
mmc_clf.fit(X_train, y_train)
mmc_clf.score(X_test, y_test) # 0.48
```

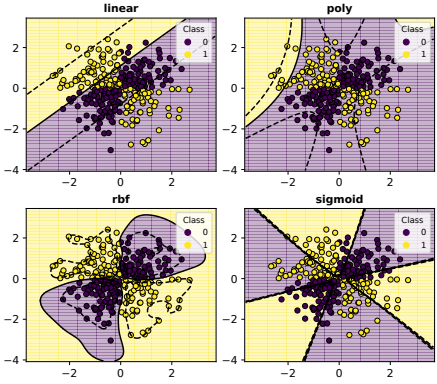


- **RBF** (default in `SVC`) achieves **0.92**; γ controls local support-vector radius.

Why Kernels Matter: Non-Linear XOR Dataset

Core Code (4-Quadrant XOR Problem):

```
 np.random.seed(0)
X = np.random.randn(300, 2)
# True when signs of x1 and x2 differ
y = np.logical_xor(X[:, 0] > 0, X[:, 1] > 0)
_, ax = plt.subplots(2, 2, figsize=(8, 8))
args = dict(long_title=False, support_vectors=False)
for k, a in zip(
  ["linear", "poly", "rbf", "sigmoid"], ax.flat):
  plot_training_data_with_decision_boundary(
    k, a, **args)
```



Key Comparison:

- Opposite quadrants share a label $\Longrightarrow$ **Linear** and **Sigmoid** completely fail.
- Odd-degree **Poly** ($d = 3$) only partially bends; **RBF** cleanly carves out all four quadrants!

Practical SVM Checklist & Assignment 3 Guide (Part 3)

1. Feature Scaling is Mandatory for SVM!

- **Trees / RF / Boosting:** Split thresholds $x_j \leq t$ only depend on feature ordering $\Longrightarrow$ scale-invariant.
- **SVM:** Margin $\|\mathbf{w}\|$ and RBF distance $\|\mathbf{x}_1 - \mathbf{x}_2\|^2$ are scale-sensitive! Unscaled features with large numeric ranges (e.g., `RestingBP` vs. `Age`) dominate the kernel.
- **Rule:** Always fit `StandardScaler` on `X_train` and transform `X_test` before fitting `SVC`.

2. Assignment 3 (Q4) Code Pattern:

```
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import confusion_matrix
# Q4(a-b): Scale features & fit RBF SVC
scaler = StandardScaler()
X_tr = scaler.fit_transform(X_train)
X_te = scaler.transform(X_test)
svc = SVC(C=1.5, kernel="rbf").fit(X_tr, y_train)
# Q4(c): Count support vectors
n_sv = len(svc.support_vectors_)
# Q4(d): Test-set confusion matrix
cm = confusion_matrix(y_test, svc.predict(X_te))
```

Take-Home Summary: Ensembles & SVM Cheat Sheet

Method	Mechanism	Primary Effect	Key `sklearn` Args	Scale?
Decision Tree	Recursive axis splits	Low bias, high var.	`max_depth`, `criterion`	No
Bagging	Parallel bootstrap trees	Reduces variance	`n_estimators`, `bootstrap=True`	No
Random Forest	Bagging + random features	Decorrelates trees	`n_estimators`, `max_features`	No
AdaBoost	Sequential re-weighted stumps	Reduces bias	`n_estimators`, `learning_rate`	No
GBDT / XGBoost	Sequential trees on residuals	Reduces bias	`max_depth=3`, `learning_rate`	No
Linear SVM	Max-margin hyperplane	Global linear margin	`kernel="linear"`, `C`	**Yes**
Kernel SVM (RBF)	Margin in $\phi(\mathbf{x})$ space	Smooth non-linear	`kernel="rbf"`, `C`, `gamma`	**Yes**

Next Steps

Run `ensemble_svm_tutorial.ipynb` locally and reuse the patterns above for **Assignment 3** (due Nov 26).